

Java 26 Direct Master Revision Sheet

Every single concept, command, and difference table from our setup sessions translated into plain, easy English.

1. Oracle JDK vs. OpenJDK (Eclipse Temurin)

All Java distributions compile their software from the same open-source codebase. The choice depends entirely on download requirements and licensing rules.

Feature	Oracle JDK	OpenJDK (Eclipse Temurin)
Core Engine	Uses the standard OpenJDK engine code.	Uses the standard OpenJDK engine code.
Performance	100% identical speeds and execution.	100% identical speeds and execution.
Account Required?	Yes. Forces you to create an Oracle login profile.	No. Provides direct, one-click installer downloads.
Cost & License	Free for personal use, but commercial use requires paid corporate contracts.	100% free forever for everyone under an open-source license.

2. Long-Term Support (LTS) vs. Feature Releases

Java releases versions under two distinct tracks based on how long those versions receive updates.

Track Option	LTS Release (e.g., Java 17, 21, 25)	Feature Release (e.g., Java 26)
Support Timeline	Receives regular updates and security fixes for multiple years.	Receives updates for only 6 months until the next version comes out.
Target Audience	Large companies that do not want to change their server code for years.	Developers and students who want to use the newest syntax tools immediately.
Syntax Availability	New helper rules are hidden or require extra preview setup flags.	New playground features run natively out of the box.

3. Installation Files: MSI Installer vs. ZIP Archive

Windows software can be bundled as an executable file package or a compressed storage folder.

Format	MSI Installer (.msi)	ZIP Archive (.zip)
Setup Method	Runs a guided wizard interface that installs the software automatically.	Dumps a folder of loose files onto your drive when unpacked.
System Variables	Automatically configures your Windows PATH and JAVA_HOME settings.	Requires you to type out system path strings manually in Windows.
Extraction Rule	No extraction needed. Direct execution block.	Unpacking a ZIP does NOT create an MSI file. It stays a raw file folder.

4. Installation Scope: All Users vs. Just Me

Choosing the installation scope determines where Java places its core folder structure on your hard drive.

Option	Install for All Users	Install for Just Me
Folder Location	C:\Program Files\Eclipse Foundation\...	C:\Users\<name>\AppData\Local\... (Hidden folder)
IDE Connection	VS Code and IntelliJ find this system path automatically.	Requires navigating hidden settings trees to point your IDE to the path.
Script Permissions	Works uniformly across standard and administrator terminals.	Can trigger permission blocks when run from admin terminal scripts.

Fixing Blank Run Buttons in VS Code

When swapping Java versions while VS Code is open, the code editor will lose track of your system paths. 1. Close every open VS Code window completely to reset the process environment. 2. Reopen your project folder. If the Run buttons still fail to show up, open the Command Palette using `Ctrl + Shift + P`, run the command `Java: Clean Java Language Server Workspace`, and click restart. This purges old memory tracks and forces VS Code to find your new Java 26 engine.

5. Traditional Class Files vs. Java 26 Implicit Class Files

Java 26 allows you to write single-file programs without creating standard boilerplate blocks.

Rule Item	Traditional Java Code Format	Implicit Java Code Format (Playground)
Outer Wrapper	You MUST explicitly write <code>public class Name { ... }</code> .	No class lines are allowed. Write methods and variables directly.
Package Declarations	Allowed. You use <code>package folder.name;</code> to organize files.	 Banned. Putting a package statement triggers a compilation error.
Mixing Styles	All functions must stay within the main class brackets.	Cannot place standalone code and full class blocks in the same file.

6. What the Compiler Does with Unwrapped Code

When you write code without a class wrapper, Java does not abandon its object-oriented rules. Instead, the compiler automatically adds the class wrapper for you when it builds the bytecode file.

If your source file is physically saved on your laptop as `calc.java` and contains this text:

```
int total = 0;

int add(int a, int b) {
    return a + b;
}

void main() {
    System.out.println(add(5, 5));
}
```

The Java compiler reads it and wraps everything inside an invisible class named after the file:

```
final class calc {
    int total = 0;

    calc() {
        super(); // Automatic blank constructor added by Java
    }

    int add(int a, int b) {
        return a + b;
    }

    void main() {
        System.out.println(this.add(5, 5));
    }
}
```

Why this automatic class is explicitly marked "final":

- **It Has No Type Name in Code:** Because the wrapper is invisible, the word `calc` does not exist as a usable keyword inside your text lines. Since you cannot type its name, no other file can write `class Advanced extends calc`. The compiler marks it `final` because inheritance is impossible anyway.

- **It Runs Faster:** The keyword `final` ensures no other class can ever extend this code or override its methods. The Java Virtual Machine can skip checking internal lookup tables for method modifications and run your code directly at maximum speed.

7. Traditional Java Organization: Why Packages are Used

In traditional Java development, you place a package statement at the absolute top of your code file: `package REPO.DSA.JAVA.Lec_1;`

- **Prevents Name Clashes:** It separates your code into isolated name folders. If two different files create a class named `Node`, putting them in separate packages prevents the compiler from getting confused.
- **Folder Matching:** The package path must perfectly mirror the physical folder hierarchy on your drive, with periods replacing folder slashes.

8. How to Compile and Run Packaged Files via Terminal

If your file sits inside a deep package structure, navigating into that local folder and typing `java Java26.java` will crash with a `wrong name error`. You must execute packaged files from your main workspace root folder using dot notation.

Assume your file structure looks exactly like this:

```
D:\Programming\          <-- Your Workspace Root Folder
├── REPO/
│   ├── DSA/
│   │   ├── JAVA/
│   │   │   ├── Lec_1/
│   │   │   │   └── Java26.java <-- Code contains: package REPO.DSA.JAVA.Lec_1;
```

The Wrong Way (Triggers Runtime Errors)

```
cd D:\Programming\REPO\DSA\JAVA\Lec_1
java Java26.java
```

Why it fails: The engine finds a file that claims to belong to the `REPO.DSA.JAVA.Lec_1` namespace, but you are trying to run it from a local, un-mapped standalone folder context.

The Correct Way (Step-by-Step Commands):

1. Move back to your main workspace root folder:

```
cd D:\Programming
```

2. Compile the file by path using standard folder slashes:

```
javac REPO/DSA/JAVA/Lec_1/Java26.java
```

3. Run the program from the root using dot notation (no file extensions):

```
java REPO.DSA.JAVA.Lec_1.Java26
```

9. Conflicts: Writing Two main Functions in One File

If you insert a traditional class block with a main method AND a free-floating playground main method inside the same source file (e.g., `demo.java`):

```
class PSVM {
    public static void main(String[] args) {
        System.out.println("Inside explicit class");
    }
}

void main() {
    System.out.println("Free-floating playground main");
}
```

What will run: The free-floating main function outside the class runs. The one inside `class PSVM` is ignored completely.

The Reason: The compiler processes the top-level `void main()` first and wraps the entire file inside the primary implicit wrapper (`class demo`). Your explicit `class PSVM` is treated as a minor nested inner class hidden inside it. The launcher always triggers the top-level method of the primary outer wrapper first.

10. Class-Level vs. Method-Level Access Modifiers

Access modifiers define visibility parameters. The rules change based on whether you apply them to a class container or a function method.

A. Class-Level Modifiers (Outer Containers)

Modifier State	Top-Level Class (Root Level of File)	Nested Class (Inside Another Class)
public class	Allowed. Visible from anywhere in the entire project.	Allowed. Visible from anywhere the parent class can be seen.
Default (No keyword)	Allowed. Visible only to files inside the exact same folder.	Allowed. Visible only to files inside the exact same folder.
private class	 Banned by the compiler. Top-level containers cannot be hidden.	Allowed. Restricts access entirely to lines inside that parent class.

B. Method-Level Modifiers (Functions)

Modifier State	Visibility Rules	Behavior on main Function
public void	Accessible from any other file across the project scope.	Valid. Launcher loads it smoothly.
Default (No keyword)	Visible to any other code file inside the same folder.	Valid. Allowed under modern playground rules.
private void	Completely locked down. Hidden from all other classes.	 Crashes at runtime. The external launcher runner cannot see it.

11. Creating Objects: The "new" Keyword Mechanics

- **Traditional Formats:** You can instantiate objects manually anywhere inside your code by using the explicit file class name string token (e.g., `java26 obj = new java26();`).
- **Implicit Playground Formats:** You cannot type statements like `calc obj = new calc();` inside the file because the implicit class name token is hidden from your text editor. However, because your main function lacks the `static` keyword modifier, it functions as an instance method. An instance method requires an active object in memory to execute. To fix this, the Java launcher automatically runs a constructor call and allocates an object instance behind the scenes for you:

```
calc implicitInstance = new calc(); // Secretly built by the JVM launcher
implicitInstance.main();           // Invoked as a working object thread
```

12. String Variable Declarations & Memory Allocation

Java handles string data memory layout completely differently based on how you write the assignment line.

Syntax Profile	String Literal (e.g., <code>String s = "Hello";</code>)	Using New Keyword (e.g., <code>new String("Hello");</code>)
Memory Allocation	Stored inside a shared deduplication cache called the String Constant Pool .	Allocated directly inside standard Heap Memory as a new item.
Deduplication Check	Checks if "Hello" already exists. If yes, reuses the old address pointer.	Skips cache checks. Bypasses the pool and forces a brand-new space.
Double Equals (==)	Evaluating (<code>str1 == str2</code>) returns true because they share one address.	Evaluating (<code>str1 == str3</code>) returns false because addresses are different.

13. Array Syntaxes vs. Flat Text Representations

A. Parameter Configurations: Bracket Placement

Java evaluates `String[] args` and `String args[]` as 100% identical instructions. They process into the exact same compiled bytecode array structures. `String[] args` is the modern Java standard because it groups the brackets cleanly onto the data type. `String args[]` is a legacy format supported only for C/C++ backward compatibility.

B. Structural Differences: Real Lists vs. Text

Data Feature	Real Array (<code>String[]</code>)	String Representation of an Array
Underlying Structure	A live collection listing object in memory holding multiple distinct items.	A single flat piece of plain text string data matching the characters of an array block.
Code Generation	Created using assignment loops or explicit brace groupings: <code>{"A", "B"}</code> .	Generated when passing a real array into utility calls like <code>Arrays.toString()</code> .
Loop Capability	Fully interactive. You can modify item entries or query items by index integers.	Static text data. It reads as plain characters: <code>"[A, B]"</code> . You cannot parse index slots.

Multidimensional Matrix Formatting (`String[][]`): Multi-layered array configurations function as arrays containing other arrays. The parent array slots store reference addresses that point directly to independent child array structures in memory.

14. The Flexible Launch Protocol Checklist

Modern Java runtime engines make passing command-line array parameters like `String[] args` completely optional. The launcher system queries compiled class bytes using a rigid top-down waterfall checklist to identify the application entry point:

1. `static void main(String[] args)`
2. `static void main()`
3. `void main(String[] args)`
4. `void main()`

If your code logic does not process command-line inputs, you can leave the function parameters blank (`void main()`) to eliminate unneeded code lines.